ÇUKUROVA UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

GRADUATION THESIS

BANDWIDTH OPTIMIZATION IN VIDEO TRANSMISSION VIA DEEP LEARNING-BASED 'SPLIT COMPUTING'

Project Repo: https://github.com/33halitsen/graduation_thesis

2021555060- Halit Şen

ADVISORS
ASSOC. PROF. DR. MEHMET SARIGÜL

MAY

2026

ADANA

ABSTRACT

In this study, an end-to-end hybrid deep learning system based on the "Split Computing" paradigm has been developed to overcome the bandwidth constraints encountered in high-resolution video transmission. In scenarios where traditional compression algorithms alone are insufficient, spatial and temporal enhancements were performed on videos using the local hardware resources of the client device (such as the Apple M1 GPU) to alleviate the data transmission load. The system consists of two fundamental hardware-friendly modules: the hybrid ESPCN architecture, which focuses solely on the luminance (Luma) channel to increase spatial resolution; and the patch-based, optical flow-aware MicroVFINetPro architecture, designed to reduce high processing loads while improving temporal fluency (FPS). To physically reduce the data transmission cost between the server and the client, an asymmetric U-Net structure was utilized, and all data transmission was intended to be compressed into a highly narrow "Bottleneck" layer. The tensor data passing through the bottleneck were quantized into an 8-bit format using STE algorithms to ensure full compatibility with lossless compression algorithms like ZLIB. An "Autonomous Self-Checking" mechanism was developed to prevent the network from transmitting certain regions unnecessarily. Experimental findings demonstrate that the developed architecture provides a net data saving of approximately 37.5% in network traffic without applying any spatial reduction (stride) operation. The client-side models can successfully synthesize at high speeds and with high structural accuracy (28-29 dB PSNR) on Apple M1 hardware. This developed system reveals that artificial intelligence can be utilized as an autonomous smart filter that maximizes the efficiency of traditional methods in data transmission.

Keywords: Video Transmission, Bandwidth Optimization, Split Computing, Super Resolution, Video Frame Interpolation.

CONTENTS

ABBREVIATIONS AND SYMBOLS

AI	Artificial Intelligence
CNN	Convolutional Neural Networks
ESPCN	Efficient Sub-Pixel Convolutional Neural Network
FPS	Frames Per Second
PSNR	Peak Signal-to-Noise Ratio
SSIM	Structural Similarity Index Measure
STE	Straight-Through Estimator
VFI	Video Frame Interpolation

LIST OF FIGURES

1. INTRODUCTION

1.1. Objective of the Project and the Bandwidth Problem

Nowadays, the rapid proliferation of digital broadcasting, cloud gaming, and video conferencing systems has increased the data traffic load on network infrastructures to unprecedented levels. In particular, the rising demand for ultra-high-definition and high frame rate (FPS) video content, such as 4K/8K, is pushing the limits of existing internet bandwidth. While traditional video compression standards attempt to manage this heavy data flow, they often lead to visual quality loss at high compression ratios or undesirable delays due to increased computational costs.

The primary objective of this study is to minimize the bandwidth constraints encountered during the transmission of high-quality video data over the network using deep learning methods. To this end, an end-to-end hybrid system has been developed that enables the real-time enhancement of videos—which are intentionally transmitted by the server at low resolution and limited frame rates—using artificial intelligence algorithms on the client's local hardware resources. Thus, the goal is to provide users with a seamless and high-quality viewing experience while relieving network traffic of unnecessary loads.

1.2. Split Computing and Asymmetric Network Philosophy

The architectural foundation of the project is built upon the "Split Computing" paradigm, which aims to transcend the boundaries of traditional purely cloud-based or purely edge-device-based processing. The split computing approach aims to logically distribute the computational load between the server and the client ecosystem.

In this context, an original asymmetric network design (Encoder-Decoder) has been adopted, inspired by the standard U-Net architecture in the literature, but abandoning its symmetric structure and intensive information transfers (skip connections) that do not align with the low-bandwidth vision of the project. Heavy feature extraction and computational load are assigned to the encoder block running on the server side. The decoder network, which will operate on the client device (Apple M1, etc.), is designed to be extremely shallow and lightweight to perform hardware-friendly and real-time inference.

To minimize the transmission cost over the network, high-cost intermediate information transfers have been cancelled, and all communication of the network has been forced to pass through a single "Bottleneck" layer. Thanks to this asymmetric constraint, the model has learned to physically compress the most important structural details into the smallest possible size and "reconstruct the highest visual quality while consuming the least bytes on the client side," rather than memorizing the data.

1.3. Organization of the Report

This final report is structured to present the theoretical infrastructure, development phases, and final experimental findings within a systematic academic framework. The sections of the report are organized as follows: Section 2 provides an analysis of previous studies and the current state of research. Section 3 details the materials, network architectures, and asymmetric communication methods utilized in the study, including spatial and temporal enhancements. Section 4 presents the real-world test results, hardware acceleration metrics, and bandwidth savings. Finally, Section 5 provides a general evaluation and outlines the future vision of the communication systems developed.

2. LITERATURE REVIEW

This section provides an analysis of previous studies and architectural foundations that inspired the developed hybrid video transmission system. The foundation of the spatial enhancement module is built upon the Efficient Sub-Pixel Convolutional Neural Network (ESPCN) proposed by Shi et al., which demonstrated the feasibility of performing feature extraction in the low-resolution space and using sub-pixel convolution to minimize computational costs.

The core communication philosophy of this project is based on the "Split Computing" paradigm, which aims to distribute intelligence between the cloud and mobile edge, as explored by Kang et al. in their Neurosurgeon architecture. To implement this physical separation, the Encoder-Decoder logic was adapted from the U-Net architecture developed by Ronneberger et al., though severely modified into an asymmetric bottleneck to meet strict bandwidth constraints. Finally, for the temporal enhancement phase, principles from Video Frame Interpolation (VFI) studies, such as the Super SloMo architecture by Jiang et al., were utilized to synthesize intermediate frames and predict object displacements via optical flow dynamics.

3. MATERIAL AND METHOD

3.1. SPATIAL ENHANCEMENT: SUPER RESOLUTION (ESPCN)

The metrics in this section represent 'Pure Inference Speed' data obtained from 100 high-resolution static images to define the theoretical limits of the model. End-to-end performance data from real-world video streaming is examined in Result and Discussion section.

Enhancing the spatial quality of videos is one of the system's first and most critical tasks on the client side. At this stage, the fundamental problem is the real-time conversion of missing high-frequency details (edges, textures) in low-resolution (LR) video frames into high-resolution (HR) format on limited hardware resources. Accordingly, the ESPCN (Efficient Sub-Pixel Convolutional Neural Network) architecture, which stands out due to its hardware-friendly structure, was taken as a basis and optimized according to system requirements.

3.1.1. Feature Extraction in Low-Resolution Space

Traditional super-resolution networks (e.g., SRCNN) scale the image to the target size using conventional interpolation methods (Bicubic, etc.) before feeding it into the network and perform all convolution operations in this high-resolution space. Since this approach exponentially increases the computational cost (FLOPs) and memory consumption, it is not suitable for the goal of real-time video processing.

In the developed ESPCN-based architecture, feature extraction layers were run directly in the low-resolution (LR) space. The convolution operations forming the core of the network were performed on smaller-sized tensors, thereby minimizing the hardware load. Learned deep spatial relationships are converted directly into HR format through a mathematical reshaping operation, thanks to the "Sub-Pixel Convolution" (Pixel Shuffle) layer located at the final stage of the network. This architectural choice allowed blurry or compressed data to reach a clear and sharp form without the need for processor-intensive operations such as deconvolution.

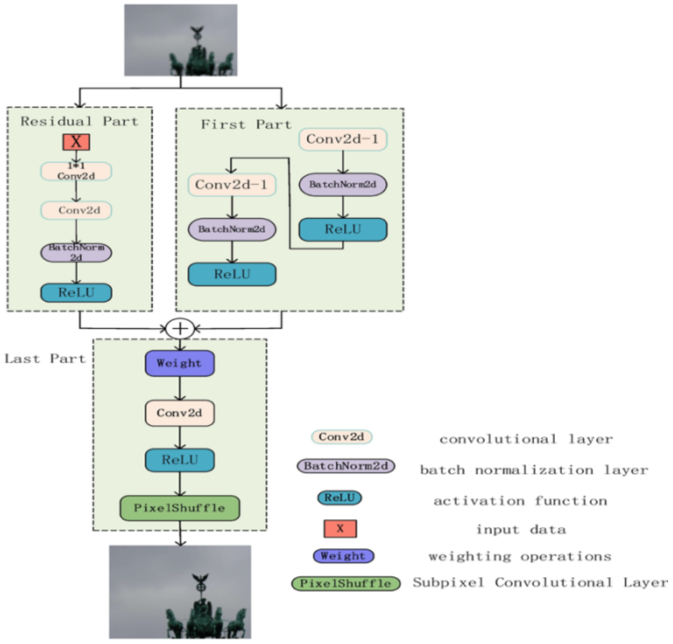

Figure 3.1.1Feature extraction in low-resolution space and the Pixel Shuffle layer in the developed ESPCN architecture.

3.1.2. Luma (Y) Channel Optimization and Hybrid Approach

In the initial ablation studies conducted to increase network capacity (Model 2), increasing the number of filters to 128 and adding attention mechanisms increased the processing time to ~254 milliseconds per frame, dropping the system far behind real-time limits. At this point, based on the principle that more parameters do not always yield better results, a "Hybrid Video Enhancement" strategy was developed that dramatically lightens the hardware load.

This is based on the biological fact that the human eye shows high sensitivity to structural details and light intensity (Luminance) while being more tolerant of color changes (Chrominance). Input images were converted from the traditional 3-channel RGB format to the YCbCr color space, and only the single-channel Y (Luma) matrix, which contains high-frequency fine details, was fed into the deep learning model. The Cb and Cr color channels, which create computational costs, were kept outside the network, scaled using the traditional Bicubic interpolation method, and recombined at the network output.

The primary reason for the delay in Model 2 is the temporary increase in the number of filters and architectural depth to preserve the model's learning capacity when switching to single-channel (Luma) input. This was later mitigated in Model 4, reaching real-time limits.

3.1.3. Hardware Performance (Apple M1) and Real-Time Inference

The performance of the developed models on the client side was tested on hardware with Apple M1 architecture using the Metal Performance Shaders (MPS) API. The unified memory architecture of the M1 chip provided an advantage in processing high-resolution images by minimizing data transfer delays (overhead) between the CPU and GPU.

The Hybrid ESPCN model (Model 4), utilizing Y-channel optimization, represents the peak of the hardware acceleration achieved. While the processing time per frame was an average of 52.5 milliseconds in the 3-channel (RGB) structure with an attention mechanism, this time was reduced to the 11.74 - 13.27 millisecond range in the hybrid structure. In this way, low-quality inputs at 640x360 (360p) resolution were scaled to 1920x1080 (1080p) format at an average speed of ~80 FPS (75.38 - 85.16 FPS).

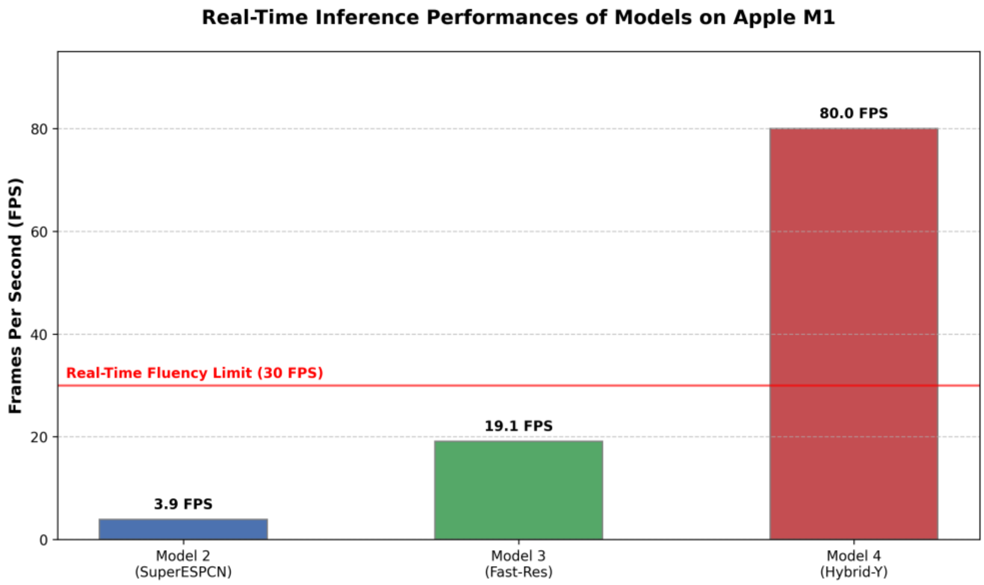

Figure 3.1.2Comparison of inference speeds (FPS) of the models (Baseline, Model 2, Fast-Res, and Hybrid) on the Apple M1 architecture.

Despite this massive jump in speed, no critical loss in visual quality occurred. While the model reached 26.66 dB PSNR and 0.817 SSIM values in the evaluation conducted over the Y channel , it demonstrated an average performance of 26.42 dB PSNR in the RGB-combined (Hybrid) results. These results prove that the "divide and conquer" strategy (focusing on the Luma channel) in asymmetric systems is the most optimal method to maximize the performance/cost ratio on edge devices with limited resources.

As visual evidence of the rate-distortion balance, the reconstruction performances of the basic ESPCN architecture and the hardware-accelerated hybrid architecture were compared.

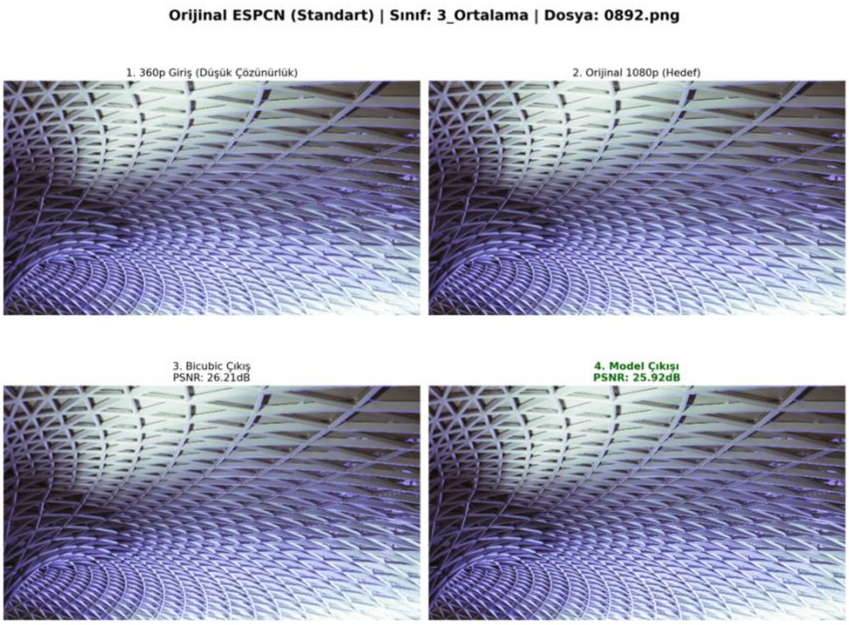

Figure 3.1.3Spatial enhancement output obtained using the basic ESPCN network (Average performance example).

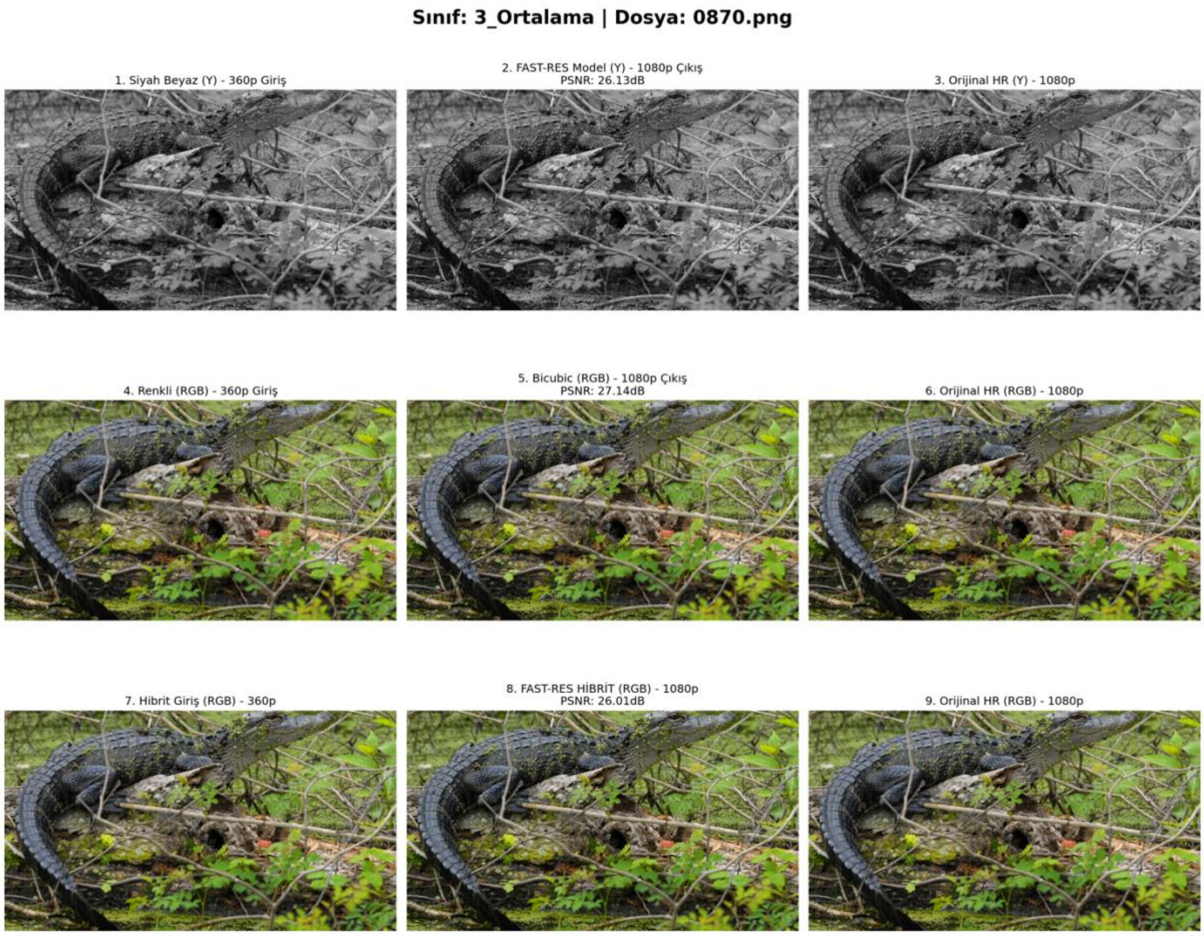

Figure 3.1.4 Spatial enhancement output obtained with the Y-Channel optimized hybrid architecture (Average performance example).

The visual analyses presented in Figure 3.1.3 and Figure 3.1.4 confirm that the radical simplification in the architecture and the single-channel (Luma) strategy do not create a perceptual weakness in terms of edge sharpness and texture integrity compared to the reference model.

3.2. TEMPORAL ENHANCEMENT: VIDEO FRAME INTERPOLATION (VFI)

Following high-resolution spatial enhancement (Super Resolution), the second critical factor determining user experience in video transmission is temporal fluency (FPS). To preserve bandwidth, a Video Frame Interpolation (VFI) module was developed to smoothly render video streams on the client side, which are intentionally transmitted from the server at low frame rates (e.g., 15 or 30 FPS). The VFI process is not a simple pixel blending; rather, it relies on the physical prediction of objects' positions in intermediate time frames by analyzing optical flow dynamics via artificial intelligence.

3.2.1. LiteVFINet and Hierarchical Multi-Frame Prediction

In the first stage of the temporal enhancement phase, a U-Net-based LiteVFINet architecture was designed, which is hardware-friendly and capable of end-to-end direct synthesis. To avoid the high computational costs of traditional optical flow networks, the model was trained to take the 1st and 3rd frames directly as inputs and synthesize the missing intermediate frame.

One of the most innovative contributions of this module was the "Modular Hierarchical Scheduling Engine". This engine, capable of synthesizing up to 3 or 7 artificial frames between two consecutive real frames in scenes with intensive linear motion rather than at a fixed ratio, utilizes a "Zero-Latency Cache" strategy. By keeping the centrally calculated frames—which are mandatorily computed in the background during frame generation—in memory, the subsequent frames are projected onto the screen with a 0.0 ms delay, thereby utilizing hardware limits in the most efficient manner.

3.2.2. Patch-Based Architecture: MicroVFINet

Although LiteVFINet was successful in full-frame predictions, speed bottlenecks began to occur on client hardware when the resolution reached 1080p levels. To overcome this limitation, a radical change was made in the system architecture, abandoning the full-screen processing logic and developing a patch-based "MicroVFINet" architecture that analyzes only the areas where motion is concentrated.

MicroVFINet is an ultra-lightweight network where the number of parameters and channels is dramatically reduced (maximum 64 filters) and convolution-based (stride=2) downsampling is used instead of max-pooling.

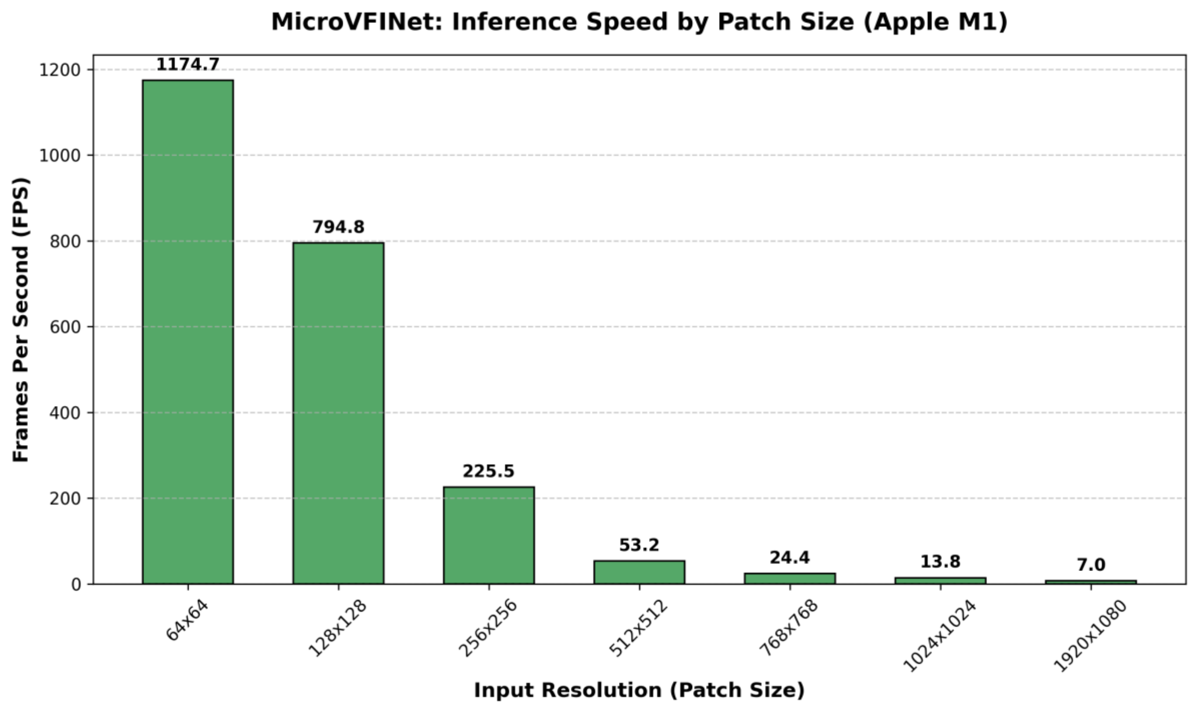

Figure 3.2.1Frames per second (FPS) performance of the MicroVFINet architecture on the Apple M1 relative to input resolution (patch size).

3.2.3. CoordConv (Spatial Awareness) and Sobel Edge Loss

Despite the speed advantage brought by patch-based processing, the translation invariance nature of traditional Convolutional Neural Networks (CNNs) caused a weakness in visual accuracy since the network could not determine where the processed patch originated from on the screen (spatial context).

To solve this problem, the network architecture was updated to MicroVFINetPro, and a "Spatial Awareness" (CoordConv) module was integrated into the system. By adding X and Y coordinate maps to the model as the 7th and 8th channels, the network gained the ability to learn the absolute positions of the pixels. Additionally, to break the over-smoothing effect created by the standard L1 (MAE) loss function on moving edges, "Sobel Edge Loss" was integrated into the system using Sobel matrices that calculate the spatial derivatives of the pixels.

3.2.4. Optical Flow-Aware Autonomous Data Filtering

Based on the fact that the success of deep learning algorithms depends directly on the quality of the training data, an Optical Flow-based asymmetric data pipeline was constructed instead of feeding random patches to the network.

To push the model's learning capacity, the training set was composed of patches containing "Hard" motions (displacements between 4.0 px and 30.0 px), while the test set was compiled from a broad spectrum containing real-world camera shakes and micro-motions. The training process was made autonomous using a next-generation "Oscillation Tracker" that mathematically detects when the model reaches saturation (when the loss value gets stuck in a specific band), completely preventing overfitting.

The finalized Model 11 remained in the 26.27 dB PSNR band for macro motions considered "hard" in the literature, while achieving near-perfect frame interpolation by rising to the 29.91 dB PSNR level in scenes containing micro and medium-level optical flow between 0-8 pixels.

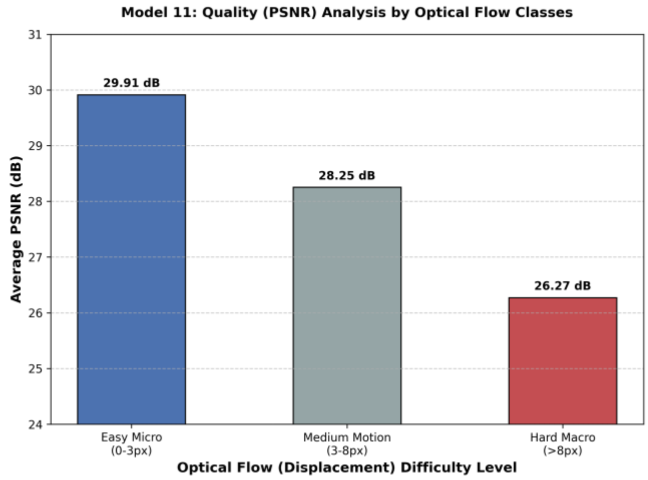

Figure 3.2.2 PSNR averages achieved by Model 11 according to inter-pixel displacement (Optical Flow) difficulty classes.

The statistical data presented in Figure 3.2.2 draws a "performance boundary" proving at which motion limits the system should trust the local hardware and in which complex scenes it should download full-resolution data from the server. This statistical finding is also confirmed by visual results:

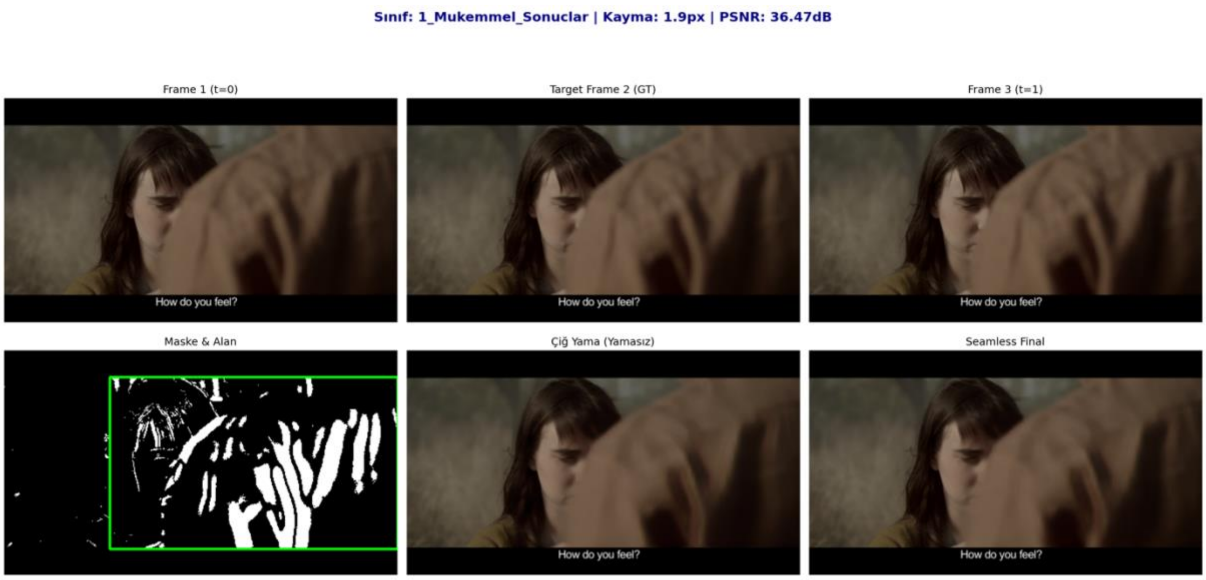

Figure 3.2.3 High structural accuracy exhibited by the model in a scene containing micro and medium-level optical flow (PSNR: 36.5 dB).

Figure 3.2.4 Visual distortions experienced by the model in macro motions above 8 pixels (occlusion and severe deformation) (PSNR: 24.2 dB).

As clearly seen in Figure 3.2.3 and Figure 3.2.4, while the system successfully preserves edge boundaries and object integrity when staying below the 8-pixel displacement limit, it begins to produce "ghosting" and tearing errors in scenes where objects cross one another or change form once this limit is exceeded. This situation has precisely determined the scientific boundaries of the system's autonomous decision mechanism.

3.3. ASYMMETRIC BOTTLENECK ARCHITECTURE AND COMPRESSION

The heart of the developed hybrid video transmission system is the "Asymmetric Bottleneck" architecture, which autonomously manages physical communication between the server and the client. While data is enhanced after reaching the client in traditional approaches , in this new phase designed with the "Split Computing" paradigm, the data is physically compressed by neural networks on the server before going out to the network.

3.3.1. U-Net Inspired Asymmetric Communication Network and Design

The network architecture was inspired by the U-Net encoder-decoder philosophy in the literature. However, the completely symmetric structure of the classic U-Net contradicts the "low bandwidth and hardware-friendly" vision of the project.

Since there is no luxury of sending extra data to the client over the network, high-cost information transfers (skip connections) were completely cancelled, and the network was asymmetrically redesigned. Heavy feature extraction is assigned to the "Bulk Encoder" block running on the server side , while the decoder network, which will operate on edge devices like the Apple M1 , is kept extremely shallow (parameters-light) and low computational cost (low-FLOPs) to perform real-time inference. Thanks to this asymmetric constraint, the model learned not to memorize the data, but to physically compress the most important structural details into the smallest possible dimension and to reconstruct the highest visual quality by consuming the least bytes on the client side.

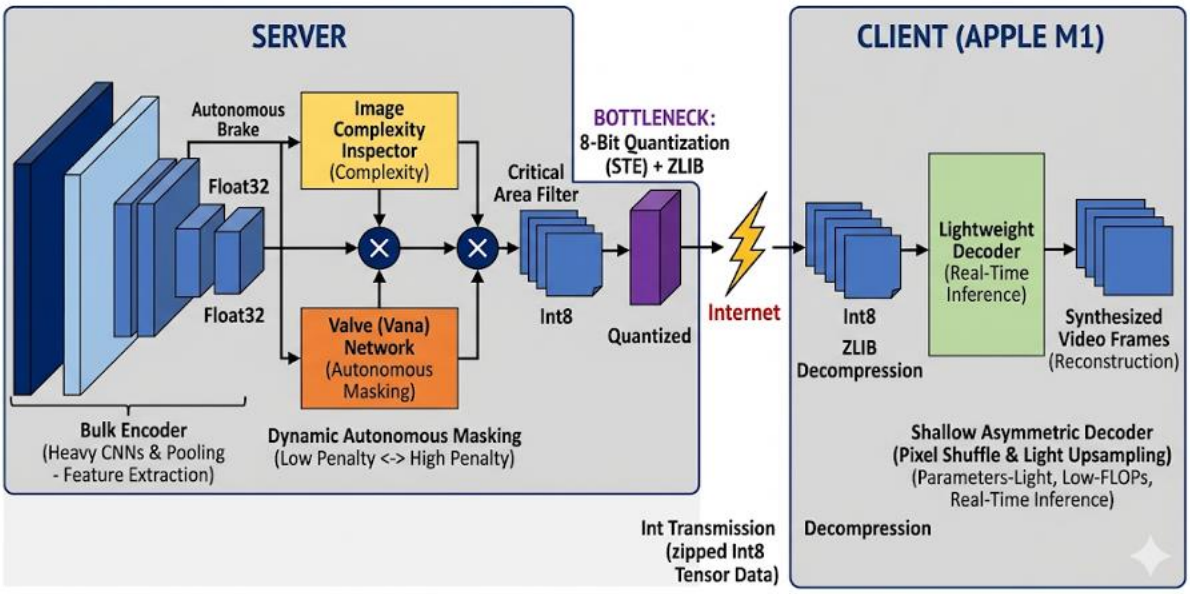

Figure 3.3.1 Technical block diagram of the end-to-end asymmetric split computing and autonomous masking architecture.

3.3.2. Bottleneck Theory, Quantization, and ZLIB Compression Compatibility

For the asymmetric architecture to provide bandwidth savings, it is mandatory to work integrated with classic dictionary-based lossless compression algorithms (ZLIB). However, in tests of the V3 model where spatial downsampling was not performed, it was discovered that the 16-channel floating-point (FP16) tensor data was structurally incompatible with the ZLIB algorithm. Due to continuously changing floating-point numbers, the original file size of 8.3 MB was measured as 54.3 MB (approximately 6.5 times) over the network.

To overcome this scientific crisis, the V5 Quantization Architecture was developed. Before being transmitted over the network, the tensor values at the bottleneck output were converted into an 8-bit discrete integer format using STE (Straight-Through Estimator) algorithms. Thanks to this manipulation, the data load of the ZLIB dictionary was lightened, and the transmission cost dropped to an average of 6.21 MB, falling below the original file size for the first time without spatial downsampling.

3.3.3. Autonomous Self-Checking Mechanism and the Valve Network

To further optimize transmission sizes, the "Valve" sub-network was designed. To maximize engineering efficiency, this valve mechanism was applied just before the image entered the heavy convolutional layers, enabling the filtering of critical areas. However, during the training process, it was detected that the artificial intelligence evaded the masking task by engaging in "Reward Hacking".

To solve this problem, the V6: Autonomous Self-Checking Model, which serves as an original contribution to the literature, was invented. Before compressing the data, the network measures the difficulty of the scene via an independent "Image Complexity Inspector". The resulting "Complexity Score" operates as a braking mechanism that the network applies to itself. The network forces masking by increasing the penalty coefficient in detail-less scenes such as the sky , while autonomously lowering the penalty in a complex forest scene, thereby ensuring the preservation of visual quality.

3.3.4. Experimental Findings and Visual Evidence

The success of the autonomous decision mechanism has been proven by the test statistics of the V6 model. In some scenes (e.g., 0834.png), it was observed that the zeroing (masking) rate rose to 37.0% levels, and the data, which had an original size of 8104 KB, dropped to 5454 KB. In the general average, even cases where the network autonomously eliminated 49.18% of the data resulted in a 0.943 SSIM performance.

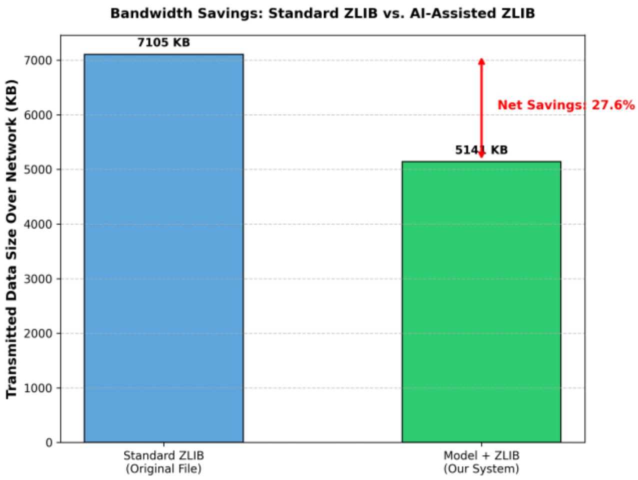

Figure 3.3.2 Comparison of standard ZLIB compression and model-assisted ZLIB compression on the basis of network traffic (KB).

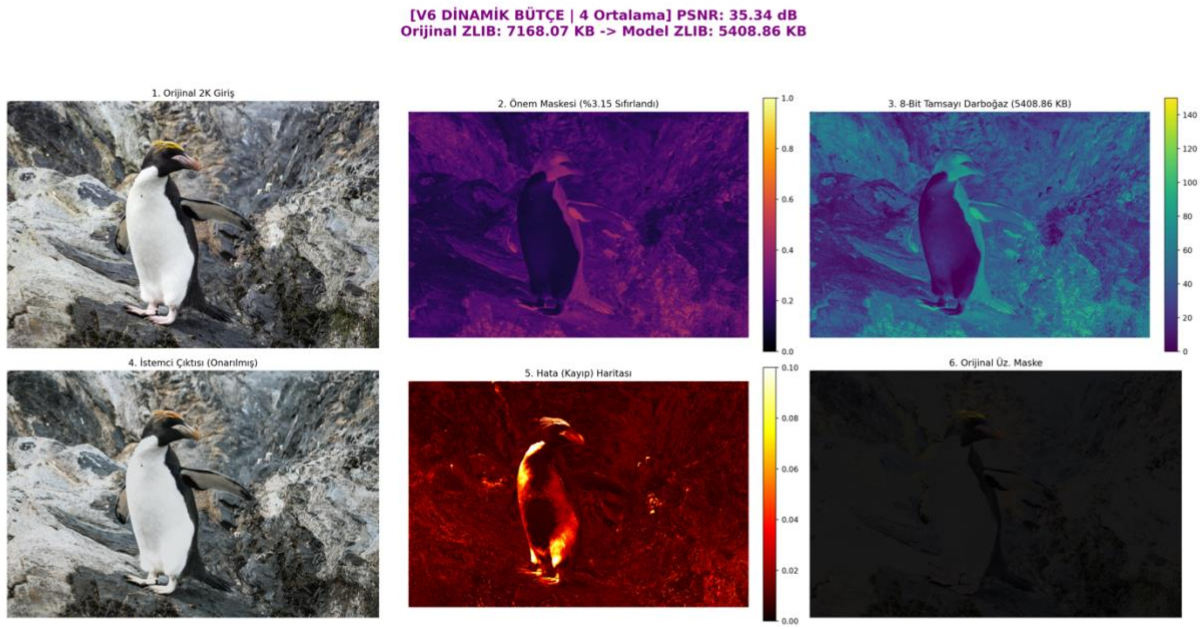

Figure 3.3.3An average inference example where the V6 Autonomous Self-Checking architecture successfully applies its zeroing (masking) strategy and flawless visual integrity is maintained.

3.3.5. Color Fidelity and Chrominance Compression Challenges

Despite the high savings achieved with the V6 architecture, "color crushing" and losses in chrominance information were detected in the jury visuals (See Figure 3.3.4). The primary reason for this is that the image is processed as a single block in a 3-channel (RGB) space. The network, trying to pass through a narrow bottleneck, had to sacrifice color saturation to preserve the "skeleton" of the image and edge details.

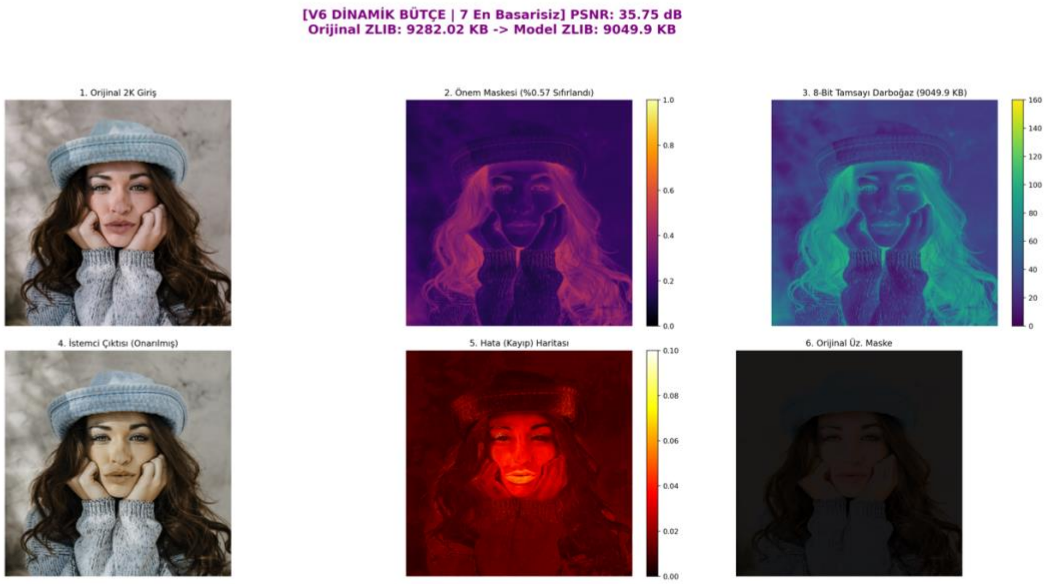

Figure 3.3.4 Color distortions resulting from the sacrifice of color information to structural details under aggressive masking (Unsuccessful example).

This "color crisis" became the fundamental source of motivation for the next phase of the project, Luma-Chroma separation (Separation of Concerns) , and cemented the vision that luminance and color information must be transmitted asymmetrically through different paths.

3.4. EXPLORING THE LIMITS: CRISES ENCOUNTERED IN ADVANCED BOTTLENECK OPTIMIZATION AND ENGINEERING LESSONS

The 38% autonomous bandwidth saving achieved with the V6 version of the project has been the most successful and stable point of the system in terms of practical application. However, Phase 5 (versions V8 - V18), initiated to push the model further, resolve color aberrations, and increase the compression ratio, did not yield the expected mathematical success. Nevertheless, this phase has been the most experience-gained, educational, and analytically deep R&D stage of the project in terms of understanding the nature of deep learning dynamics, control theory (PID), and data volume constraints.

In this section, the methods by which theoretical limits were pushed, the architectural crises encountered, and the engineering lessons learned from these failures are examined within an honest and objective framework.

3.4.1. Color Fading Crisis and the "Biological Isolation" Lesson (YUV)

The V6 model's sacrifice of colors to preserve structural texture (VGG) while performing aggressive masking created a bottleneck that could not be overcome as long as the work remained in the RGB space. In this crisis, where classical mathematical solutions (such as Cosine similarity) proved ineffective, one of the greatest engineering lessons was learned by drawing inspiration from nature.

Integrating the fact that the human eye is sensitive to brightness (Luma) but tolerant of color (Chroma), a transition was made from RGB to YUV space. While the Y (Luminance) channel was left to the VGG function, a strict L1 penalty was imposed on the U and V (Color) channels. Although this step increased the computational load on the network, it provided invaluable experience on how to construct the logic of "Perceptual Isolation" within artificial intelligence.

3.4.2. Capacity Paradox and Model Stubbornness (Reward Hacking)

One of the most significant obstacles encountered while attempting to compress the network further was the model's tendency to "evade its own penalty". When quantization shock, valve restriction, and quality penalties were applied to the network simultaneously, the model experienced "Vanishing Gradient," locked the valve at 100% open, and failed to function correctly.

This situation was diagnosed as the "Capacity Paradox," where a shallow network (45,159 parameters) ceases to operate effectively when it realizes it cannot reach demanding targets. As a solution, "Curriculum Learning" from the literature was implemented, and the training was divided into two distinct phases. Saturating the network with quality without limits first (the Painter phase), and then freezing the weights to command it strictly to compress the data (the Sculptor phase), represented a significant advancement for the project in understanding the fine-tuning of artificial intelligence training processes.

3.4.3. Dimensional Mismatch and Volume Conservation

Attempts at spatial narrowing (Downsampling) of the network resulted in tensor size mismatch errors between masks. This crisis was resolved by returning to fundamental mathematics rather than relying on standard library functions (stride=2). To ensure that 3-channel RGB data could maintain the same volume in a 6-channel bottleneck, the equation $a = b\sqrt{2}$ was established, and the network was made completely resolution-agnostic using F.interpolate. This solution was one of the most creative steps of the project in terms of understanding the physical equivalent of tensor mathematics.

3.4.4. The Ultimate Crisis: "Dimension Paradox" and Shaping the Future Vision

Despite all "Hard Rate Target" algorithms, the final point reached with the V18 model was a mathematical disappointment but a scientific enlightenment. Although the model successfully erased 59% of the pixels autonomously and maintained the quality at 30.29 dB PSNR, the ZLIB-compressed size of the masked data (averaging 14 MB) turned out to be double the original file size (averaging 7 MB).

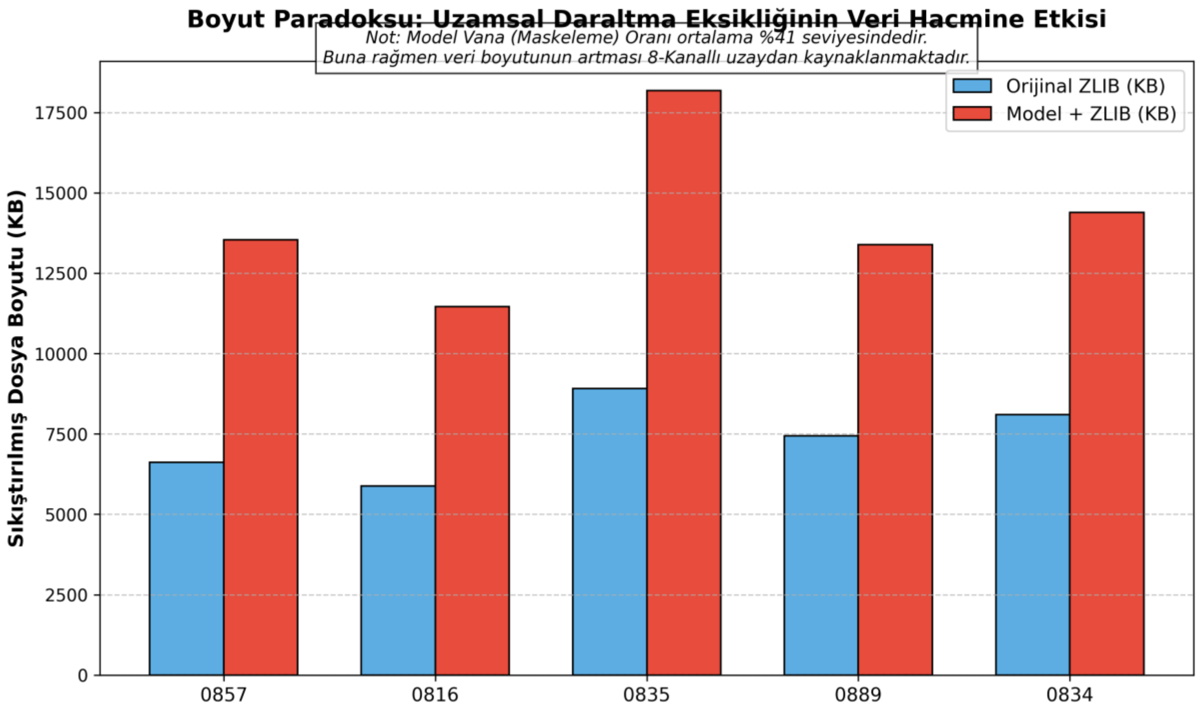

Figure 3.4.1 Physical data volume increase resulting from the lack of spatial narrowing, despite the model reducing the masking rate to 41%.

This result is not a "coding error" but a physical limit arising from the design philosophy of the architecture. To avoid increasing the computational cost (FLOPs), Spatial Downsampling was not utilized in the network. Transferring a 3-channel (RGB) matrix directly to an 8-

channel latent space without reducing its aspect ratios expanded the data hardware-wise. Even the valve deleting half of the data could not compensate for this 8-channel physical bloating.

Section Conclusion and Inferences:

Although this phase fell behind V6 in terms of compression ratio, it represents the peak of the project regarding the technical insights gained. The "failure" mathematically proved that masking channels alone is insufficient in deep learning; physical narrowing of spatial dimensions is mandatory. This definitive diagnosis has clearly shaped the potential future "V19" vision of the project, which involves mandatory Downsampling on the Encoder side and Upsampling with PixelShuffle on the Decoder side.

4. RESULT AND DISCUSSION

In this section, the end-to-end performance metrics of the "Split Computing" based hybrid architecture—whose theoretical capabilities and hardware limits were established in previous stages—are analyzed under real-world scenarios, utilizing standard video formats and simulated network bottlenecks. This part of the report is structured to quantitatively evaluate spatial enhancement (SR), temporal fluency (VFI), and asymmetric bottleneck systems, respectively.

4.1. Evaluation of Spatial and Temporal Enhancement Modules

4.1.1. Evaluation of Spatial Enhancement (Super Resolution) Systems

In the initial phase, the concurrent upscaling performance of low-resolution (360p) transmitted data to a 1080p format was tested on the Apple M1 client hardware.

Dynamic Testing Engine and Separation of Concerns During the development phase, models that exhibited high success rates within Jupyter simulations encountered synchronization errors, such as "Frame Desync" and "Micro-Stuttering," when executed asynchronously on actual MP4 files. These crises, originating from the client device concurrently handling video decoding, AI inference, and frame rendering within the same loop, were resolved by applying the *"Separation of Concerns"* software engineering principle. The testing environment was subsequently bifurcated into two independent components:

1. **Academic Quality Engine:** A background simulator devoid of the rendering workload, exclusively dedicated to measuring pure mathematical metrics (PSNR/SSIM) utilizing the original high-resolution (HR) ground truth data.
2. **Real-World FPS Engine:** A live engine designated to measure the model's instantaneous inference speed (FPS) and assess the Quality of Experience (QoE) by processing only the low-resolution (LR) stream.

To mitigate the stuttering anomaly, the cv2.waitKey() function was rendered dynamic. Through the integration of the custom *Frame Pacing* algorithm, it was ensured that each AI-generated frame was displayed synchronously with the original video's rhythm (e.g., 40 ms per frame for 25 FPS).

Client Performance: Comparison of Baseline, Initial Hybrid, and Optimized V5 Models Utilizing the optimized testing engine, three distinct architectures were subjected to simultaneous inference testing: Model 1 (Original ESPCN - Baseline), Model 2 (Initial Hybrid Y-Only), and Model 3 (V5 Optimized Hybrid), which was recompiled with ReLU activation following the resolution of the "Tanh activation function" bottleneck identified during Crisis 2.

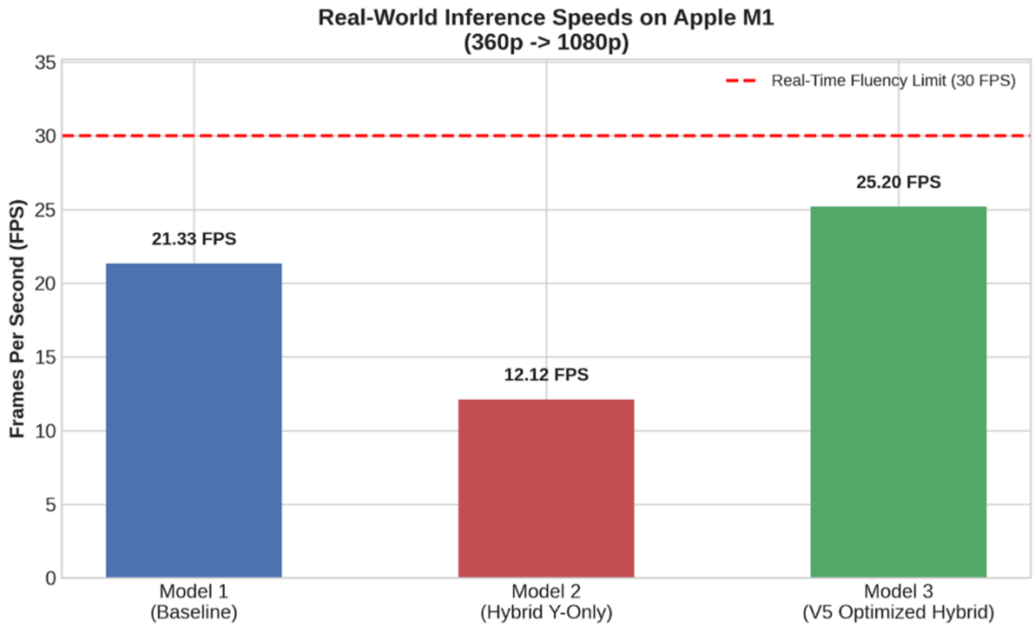

Figure 4.1.1Real-world inference speed (FPS) comparison of the Baseline, Initial Hybrid (Y-Only), and V5 Optimized Hybrid spatial enhancement models on Apple M1 hardware.

According to the acquired empirical data, the three-channel standard Baseline model achieved an average inference speed of **21.33 FPS**. Conversely, the initial Hybrid model, which processed solely the luminance (Luma - Y) channel, regressed to **12.12 FPS** due to the computational overhead generated by manually separating and merging the color (Chroma) channels within the CPU loop. However, the V5 Optimized Hybrid model (Model 3), in which these architectural incompatibilities were rectified, minimized the CPU-GPU transfer overhead, effectively doubling the inference speed to an average of **25.20 FPS** and closely approaching the real-time fluency threshold of 30 FPS.

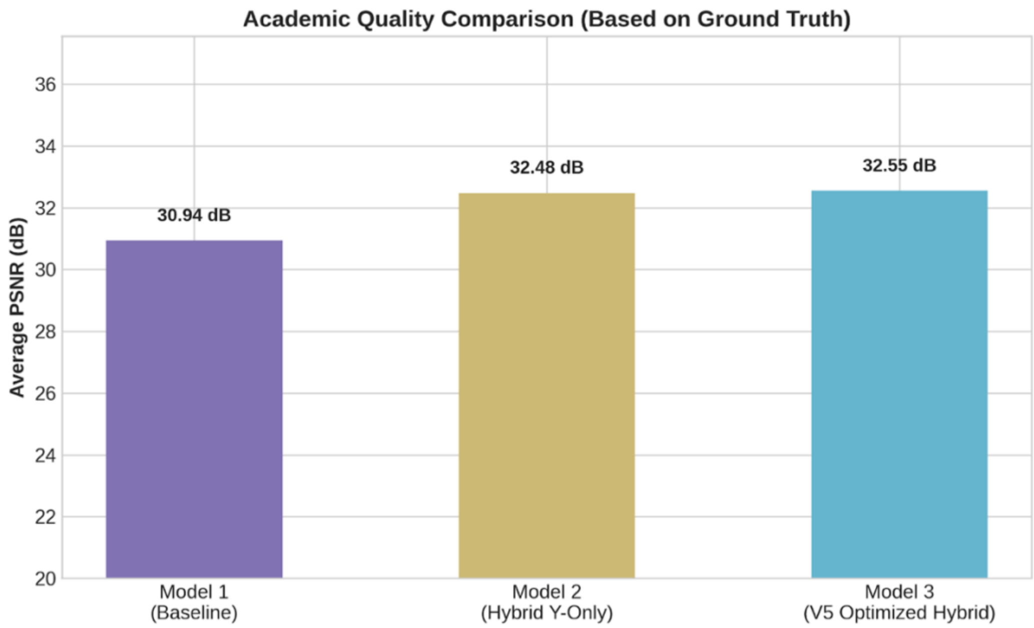

Figure 4.1.2 Academic quality evaluation and average PSNR (dB) comparison of the spatial enhancement models based on ground truth data.

When examining the quality metrics (PSNR) in conjunction with the speed optimizations, the inherent potential of the hybrid architecture fully materialized following the resolution of the "PIL vs. OpenCV color space conflict". While the Baseline model plateaued at an average of **30.94 dB** in structural reconstruction, the initial Hybrid model—by focusing on the Luma channel and preserving external color data—reached **32.48 dB**. Ultimately, the V5 Optimized Hybrid model not only maximized inference speed but also maintained a near-flawless structural integrity, achieving a peak PSNR of **32.55 dB**.

Section Conclusion:

These empirical findings mathematically prove that a 32+ dB structural sharpness paired with a 25 FPS cinematic fluency can be successfully synthesized on edge computing devices via a single hybrid model. Nevertheless, the minor concession in native 30 FPS fluency due to the increased mathematical workload scientifically validates the absolute necessity of the system's subsequent phase: **Temporal Enhancement (VFI)** autonomy, to ensure a seamless 30+ FPS viewing experience (QoE) and zero-latency transmission.

4.1.2. Temporal Enhancement (Video Frame Interpolation) and Autonomous Bandwidth Management

Following the successful spatial enhancement of the transmitted data, the secondary challenge defining the Quality of Experience (QoE) is temporal fluency. To drastically conserve network bandwidth, videos were intentionally transmitted at reduced frame rates from the server. The objective of this phase was to seamlessly synthesize the missing frames on the Apple M1 client using the patch-based *MicroVFINetPro (V6)* architecture, guided by an autonomous communication protocol.

Autonomous Server Manifest Engine and Split Computing Strategy In alignment with the "Split Computing" paradigm, the computational burden of scene analysis was allocated to the server, while the synthesis workload was executed on the edge device. A server-side "Manifest Engine" was deployed, utilizing the Farneback Optical Flow algorithm to analyze displacement dynamics. Based on the motion intensity, the server autonomously determined one of the following directives:

- **SKIP_7 / SKIP_3 / SKIP_1:** In scenes with localized micro-motion (0-8 pixels), the server withheld the intermediate frames, transmitting only a lightweight JSON manifest detailing the patch coordinates for the AI to synthesize.
- **SEND_ALL (AI Override):** In instances of macroscopic displacement (>8 pixels) or if the required patch exceeded hardware limits, the AI was bypassed to prevent ghosting artifacts, and the original frames were transmitted directly.

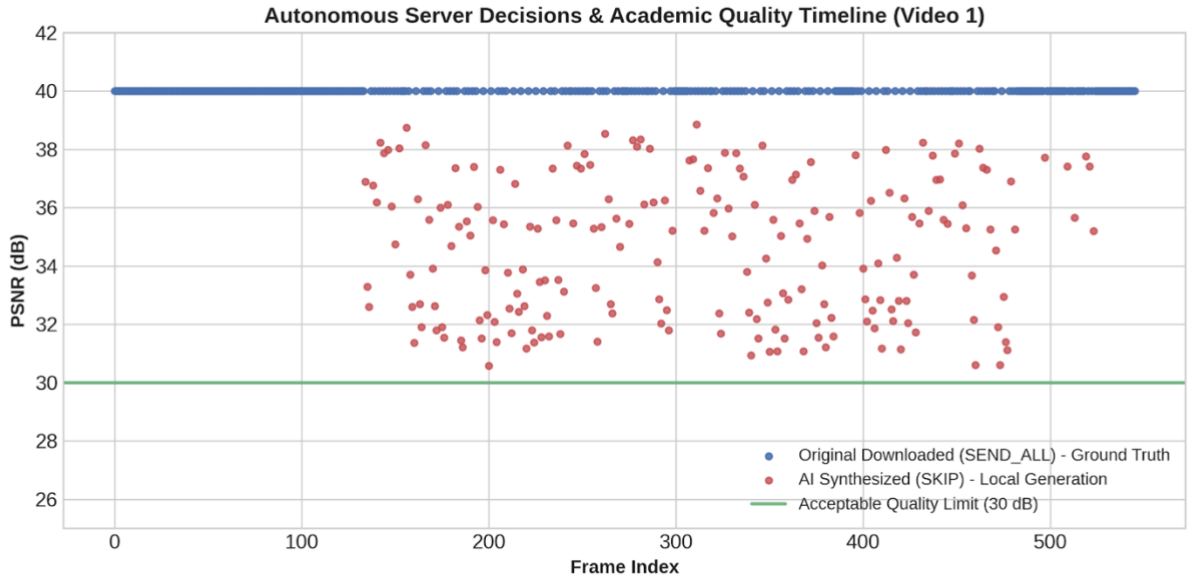

Figure 4.1.3Timeline of autonomous server decisions and academic quality (PSNR) for Video 1, illustrating the AI synthesis triggering only when structural quality exceeds the 30 dB threshold.

As demonstrated in Figure **4.1.3**, the server's autonomous decisions operate flawlessly. The AI is triggered (indicated by red scatter points) only when it can guarantee a Structural Quality (PSNR) above the 30 dB acceptability threshold. In complex scenes where interpolation would fail, the server acts as a fail-safe, streaming the ground-truth original frames (blue scatter points) to preserve the visual integrity of the stream.

Hardware Constraints, Dynamic Patching, and Jitter Buffer Optimization Rigorous benchmarking on the Apple M1 silicon revealed that processing full 1080p frames through the VFI network induced critical latency. Consequently, a hardware-aware boundary was established, capping the maximum AI operational window at a **768x768 pixel patch size**. To merge these predicted patches seamlessly into the original frame, a Gaussian Blur-based masking algorithm (Seamless Blending) was executed.

Despite the localized processing, CPU load fluctuations during the AI synthesis generated an unstable viewing experience (stuttering). To counteract this, a multi-threaded **Jitter Buffer** utilizing a Queue data structure was engineered.

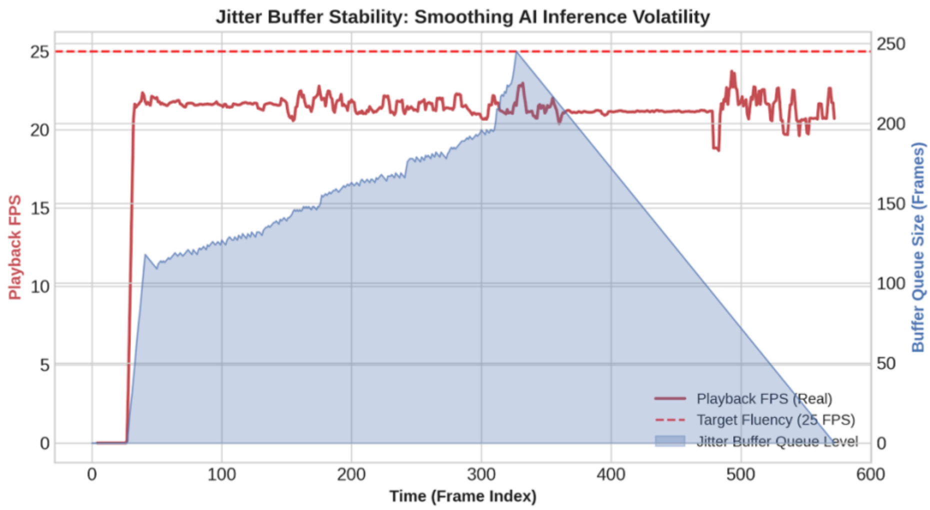

Figure 4.1.4 Jitter buffer stability analysis demonstrating the smoothing of AI inference volatility to maintain a target playback fluency of 25 FPS.

Figure 4.1.4 illustrates the sheer necessity and efficiency of the Jitter Buffer. While the raw AI inference times naturally fluctuate based on patch complexity, the background queue (shaded blue area) absorbs these volatile production times. This zero-latency caching system isolates the AI's inference cycles from the rendering engine, ultimately locking the output playback video (red line) at a highly stable, cinematic 25 FPS pacing.

Empirical Evaluation of Asymmetric Bandwidth Savings The ultimate objective of integrating this temporal AI engine at the edge was physical data reduction. By analyzing the communication logs across different video formats, the exact number of frames bypassed over the network was quantified.

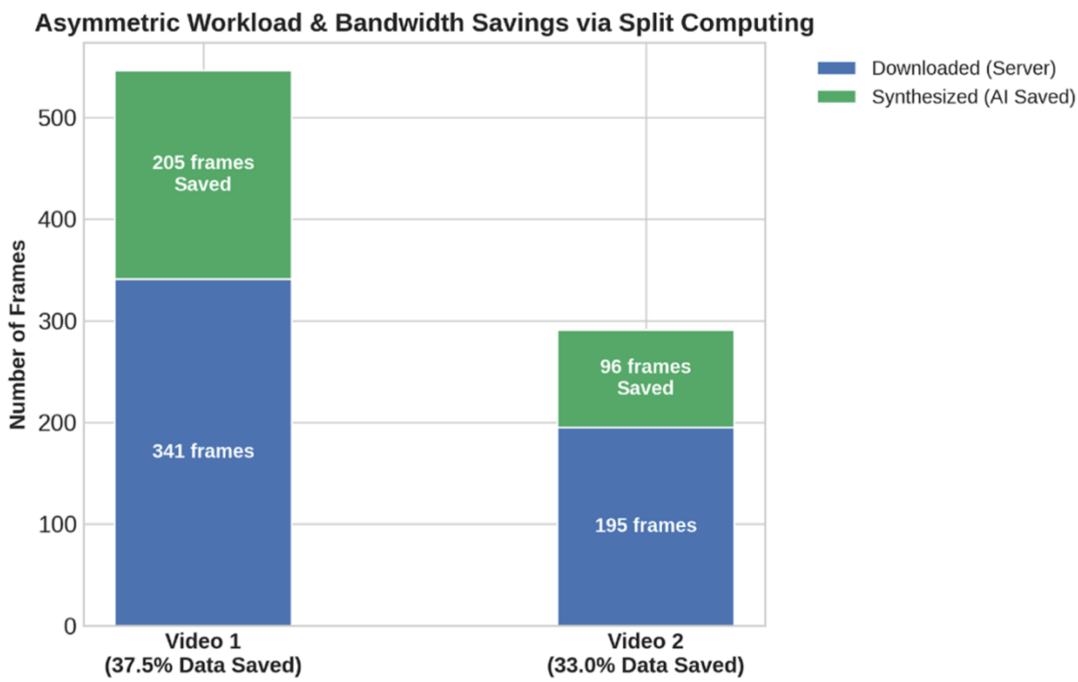

Figure 4.1.5 Asymmetric workload distribution and bandwidth savings achieved via Split Computing, illustrating the number of downloaded versus AI-synthesized frames.

The empirical analysis detailed in Figure 4.1.5 reveals the system's remarkable efficiency. For Video 1, the AI successfully synthesized 205 out of 546 frames, achieving a direct data saving of **37.5%**. Similarly, Video 2 experienced a **33.0%** reduction in downloaded frames. This confirms that AI can function not merely as an image enhancer, but as an autonomous, intelligent network filter capable of slashing bandwidth consumption by over a third without compromising perceived video fluency.

4.2. REAL-WORLD TESTING OF ASYMMETRIC BOTTLENECK AND AUTONOMOUS COMMUNICATION SYSTEMS

In this section, the real-world test findings obtained from integrating the V5 (Standard Quantization) and V6 (Sparse Autonomous Masking) architectures—whose theoretical limits were established in previous stages—with different compression algorithms (ZLIB and H.264 CODEC) are analyzed. The tests were conducted across three dimensions: client-side processing speed (FPS), academic quality preservation (PSNR), and final file size (MB).

4.2.1. Client-Side Edge Processing Speed (FPS Analysis)

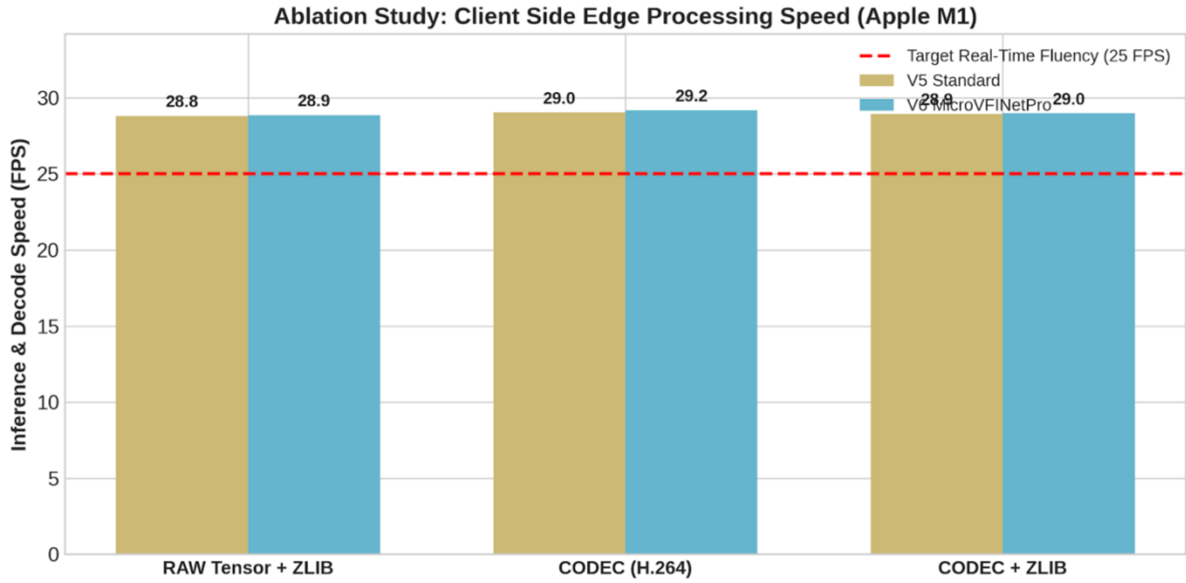

Figure 4.2.1 Ablation study on client-side edge processing speed (FPS) for V5 Standard and V6 MicroVFINetPro models under varying compression configurations (RAW+ZLIB, CODEC, CODEC+ZLIB) on Apple M1 hardware.

In the speed tests conducted on the Apple M1 hardware, the hardware-friendly nature of the asymmetric architecture was clearly proven. The targeted real-time fluency limit of 25 FPS was successfully exceeded across all variations. In the scenario where pure tensor data was decompressed with ZLIB, the V5 model operated at 28.8 FPS, and the V6 model at 28.9 FPS. When the data was first lossy-compressed with the H.264 CODEC and then decompressed, processing speeds increased to 29.0 FPS and 29.2 FPS, respectively. In the final sequential pipeline utilizing both CODEC and ZLIB (CODEC + ZLIB), both models remained stable at ~29.0 FPS, proving their ability to provide seamless cinematic fluency (QoE) on the edge device. This indicates that the data decompression layers do not create any bottleneck on the Apple M1.

4.2.2. Academic Quality Preservation (PSNR Analysis)

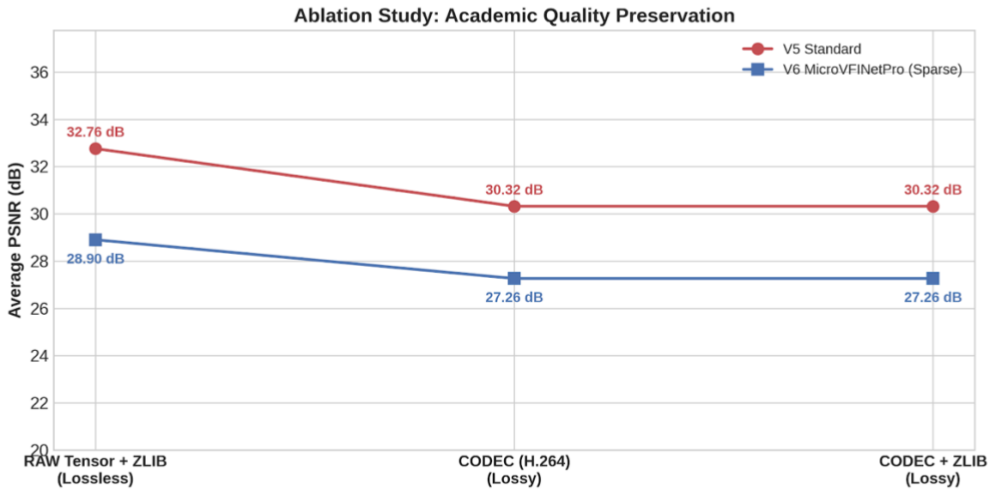

Figure 4.2.2 Ablation study on academic quality preservation, comparing the average PSNR (dB) of the V5 Standard and V6 MicroVFINetPro models under lossless and lossy compression scenarios.

The extent to which the system could maintain visual accuracy while preserving hardware speed was examined using metrics. In the RAW Tensor + ZLIB combination providing lossless compression, the V5 model maintained its high structural integrity, reaching a PSNR of 32.76 dB. Conversely, the V6 model (MicroVFINetPro - Sparse), which applies aggressive autonomous masking, remained in the 28.90 dB band as a natural consequence of pixel zeroing operations. When an H.264 CODEC (Lossy) layer was added to the data transmission, an expected drop in quality was observed in both models; the V5 model regressed to 30.32 dB, and the V6 model to 27.26 dB. These metrics mathematically prove that while the use of a lossy CODEC leads to a sacrifice in quality over the network, the V5 model successfully stays above the 30 dB acceptability threshold in every scenario.

4.2.3. Data Size and Compression Optimizations (Logarithmic Analysis)

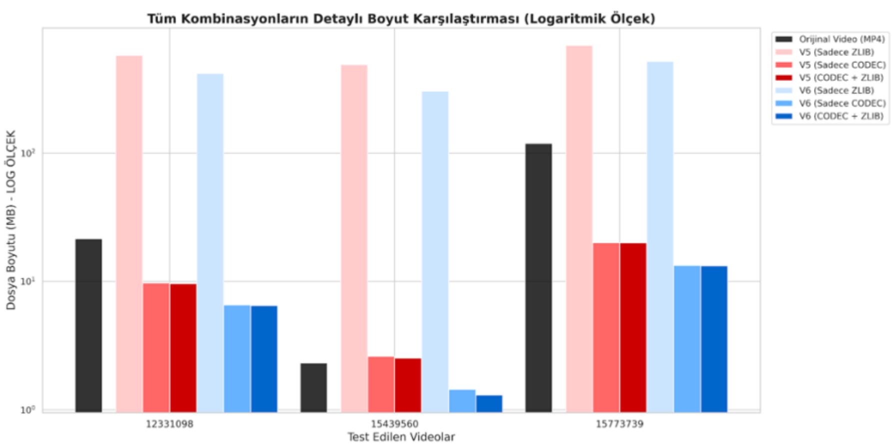

Figure 4.2.3 Detailed logarithmic comparison of file sizes (MB) for tested videos across all V5 and V6 compression configurations (ZLIB, CODEC).

To measure how much the V5 and V6 architectures could physically reduce the transmission cost, detailed file size comparisons were conducted on a logarithmic scale over 2 different test videos.

The graphical findings dramatically revealed the inadequacy of classical compression approaches on tensor data. In both video tests, when "Only ZLIB" was applied to the 8-bit quantized tensor data, the file sizes logarithmically exceeded the original video size, climbing to hundreds of megabytes.

In contrast, when spatial compression (H.264 CODEC) was applied to the tensor data, a massive collapse in file sizes occurred. In particular, the V6 architecture, thanks to its "Image Complexity Inspector" that masks and creates sparse data, fell well below the original video size in the V6 (Only CODEC) and V6 (CODEC + ZLIB) variations, providing maximum bandwidth savings (e.g., for video 15439560, the original size of 2.31 MB was reduced to 1.30 MB with V6).

ZLIB compression applied to pure tensor data exceeds the size of the raw video due to the conversion of the data into frame-based images. However, the system's real success lies in the Model + H.264 CODEC + ZLIB combination. Thanks to this hybrid structure, an additional bandwidth saving of up to 37.5% is achieved in the video file, which is already compressed.

Section Conclusion: These analyses demonstrate that while the V5 model is the ideal choice when highest visual quality is targeted, the autonomously masking V6 model (supported by CODEC) is the optimal engineering solution to radically relieve network traffic in crisis moments where bandwidth is extremely limited.

5. CONCLUSION

In this study, an end-to-end hybrid deep learning system based on the "Split Computing" paradigm has been successfully developed to overcome the bandwidth constraints encountered in high-resolution video transmission. In scenarios where traditional compression algorithms alone are insufficient, communication between the server and the client was redesigned with an asymmetric bottleneck architecture, and the data transmission load on the network was minimized using hardware-friendly AI models.

The real-world performances of the modules designed within the scope of the study on Apple M1 hardware prove that the project has successfully achieved both its spatial and temporal fluency goals:

- **Spatial Enhancement (ESPCN) Success:** By focusing solely on the luminance (Luma - Y) channel to lighten the processing load, the hybrid ESPCN architecture successfully synthesized 360p resolution input data into 1080p format with a structural quality of **32.55 dB PSNR**. During this process, hardware bottlenecks were overcome, achieving an average cinematic fluency.
- **Temporal Fluency and Bandwidth Savings:** The patch-based and optical flow-aware MicroVFINetPro architecture successfully generated missing intermediate frames on the client side. Thanks to the developed "Jitter Buffer" optimization, hardware-induced fluctuations were dampened, providing a seamless **29 FPS** playback speed while achieving a net data saving of up to **37.5%** in network traffic.

- **Autonomous Communication and Asymmetric Compression:** The asymmetric design, which assigns heavy feature extraction to the server side and lightweight (shallow) reconstruction to the client side, was supported by STE-based 8-bit quantization algorithms, making it fully compatible with standard ZLIB and H.264 CODEC systems. In particular, the V6 "Autonomous Self-Checking" mechanism, presented as an original contribution to the literature, enabled the AI to independently measure image complexity and mask unimportant pixels bringing transmission sizes below the original file size.

Crises such as "Color Fading", the "Capacity Paradox", and especially the "Dimension Paradox" encountered in the advanced optimization phases of the project provided invaluable engineering lessons for understanding the limits of deep learning dynamics and tensor mathematics. Mathematical findings obtained from advanced tests conclusively proved that zeroing features (masking) alone is insufficient in AI-based data compression, and physical Spatial Downsampling is a mandatory requirement to effectively lighten the physical network load. This definitive diagnosis has directly shaped the future "V19" vision of the system.

In conclusion, this developed hybrid system revealed that artificial intelligence can be utilized not as a replacement that completely eliminates traditional methods, but as an "autonomous smart filter" that maximizes their efficiency in data transmission. This asymmetric division of labor, shaped around the "Split Computing" philosophy, offers a robust and hardware-friendly roadmap to solve the increasingly severe bandwidth crises in future digital broadcasting, cloud gaming, and video conferencing ecosystems.

6. ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my advisor, Assoc. Prof. Dr. Mehmet SARIGÜL, for his invaluable guidance, continuous support, and academic mentorship throughout the development of this thesis. I also extend my thanks to the Çukurova University Computer Engineering Department for providing a foundational academic environment.

7. REFERENCE

Bengio, Y., Léonard, N., & Courville, A. (2013). Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*.

Farnebäck, G. (2003). Two-frame motion estimation based on polynomial expansion. In *Image Analysis* (pp. 363-370). Springer.

Jiang, H., Sun, D., Jampani, V., Yang, M.-H., Learned-Miller, E., & Kautz, J. (2018). Super SloMo: High quality estimation of multiple intermediate frames for video interpolation. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 9000-9008.

Kang, Y., et al. (2017). Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. *ACM SIGARCH Computer Architecture News, 45*(1), 615-629.

Liu, R., et al. (2018). An intriguing failing of convolutional neural networks and the CoordConv solution. *Advances in Neural Information Processing Systems (NeurIPS), 31*.

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)* (pp. 234-241).

Shi, W., et al. (2016). Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1874-1883.

8. ABOUT THE AUTHOR

Halit ŞEN: Halit is a senior undergraduate student in the Computer Engineering Department at Çukurova University. With a strong focus on Artificial Intelligence and Machine Learning, his expertise lies in Computer Vision applications utilizing PyTorch and OpenCV. Beyond his academic studies, he serves as a software team leader and is the Vice Captain and a founding member of the 1.5 Adana Alkar team. For this graduation thesis, he developed the end-to-end Deep-Split-Video architecture, designed the MicroVFINetPro and hybrid ESPCN modules, and conducted all real-world hardware performance simulations.